Lecture_09_Three_Dimensional_Viewing_and_Clipping

CSTE 3201 Computer Graphics

Lecture 09: Three-Dimensional Viewing and Clipping

> **Previous lecture connection:** Lecture 8 explained the mathematics of projection: how 3D points are projected onto a 2D plane. This lecture places that projection inside the full **3D viewing pipeline**: camera setup, view coordinate transformation, view volume, clipping, perspective divide, and viewport mapping.

Learning Outcomes

1. Explain the purpose of 3D viewing in a graphics system.
2. Distinguish between object, world, view, clip, normalized device, and screen coordinates.
3. Describe how a camera coordinate system is built from eye, look-at point, and up vector.
4. Explain orthographic and perspective view volumes.
5. Apply basic point and line clipping ideas in 3D.
6. Explain why clipping is performed before the perspective divide.
7. Map normalized device coordinates to screen coordinates.
8. Identify common mistakes in 3D viewing and clipping.

---

1. Why 3D Viewing Is Needed

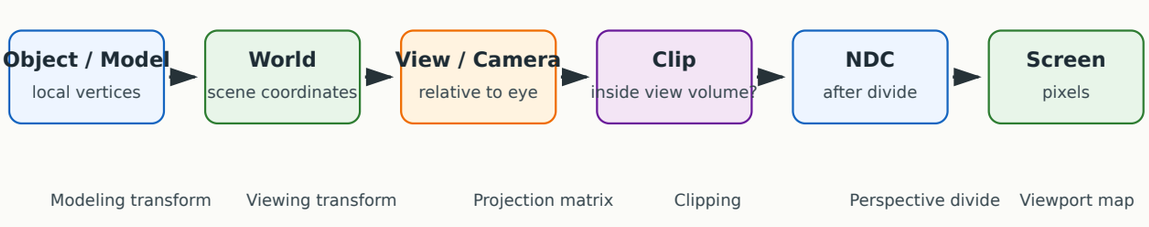
In 2D graphics, objects can often be drawn directly on a 2D screen after transformation and clipping. In 3D graphics, the situation is more complicated because the world is three-dimensional but the display is two-dimensional.

A graphics system must answer several questions:

- Where is the camera?
- Which direction is it looking?
- What part of the world is visible?
- Which objects are outside the viewing region?
- How should 3D positions become 2D screen positions?
- Which pixels should finally be drawn?

This complete process is called **3D viewing**.

Figure 1: 3D Viewing Pipeline



Main idea: 3D viewing converts a scene into the camera's coordinate system, keeps only visible geometry, and m

A useful way to think about 3D viewing is this:

> 3D viewing is the process of describing a scene from the camera's point of view, discarding what the camera cannot see, projecting the remaining geometry, and finally mapping it to screen pixels.

---

2. Coordinate Spaces in the 3D Pipeline

Modern graphics systems do not keep a point in only one coordinate system. The same point passes through several coordinate spaces.

2.1 Object or Model Coordinates

These are local coordinates used when an object is created.

Example: A cube model may be defined from `(-1,-1,-1)` to `(1,1,1)` around its own local origin.

2.2 World Coordinates

Object coordinates are transformed into a shared scene coordinate system. After this step, all objects are placed relative to one another.

Example: A table, chair, and lamp are all positioned in the same room.

2.3 View or Camera Coordinates

World coordinates are transformed so that the camera becomes the reference frame. In this coordinate system, the camera is usually placed at the origin and looks along a standard direction.

2.4 Clip Coordinates

After projection, points are represented in homogeneous clip coordinates. Clipping is usually performed in this space.

2.5 Normalized Device Coordinates

After the perspective divide, visible points are mapped to a normalized cube, often with x, y, and z values in a standard range.

2.6 Screen or Window Coordinates

Finally, normalized coordinates are mapped to actual screen pixels.

---

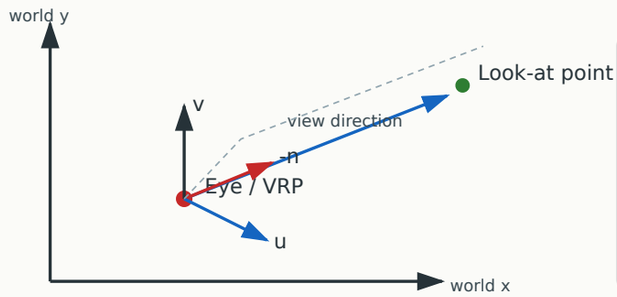
3. The Camera Analogy

A virtual camera is similar to a real camera, but it is defined mathematically. The basic camera setup usually needs three things:

1. **Eye position**: where the camera is located.
2. **Look-at point**: the point the camera is looking toward.
3. **View-up vector**: a direction that tells which way is upward for the camera.

From these, we construct a camera coordinate system.

Figure 2: Camera Setup and u-v-n View Basis



View basis vectors

```
n = normalize(eye - center)
u = normalize(VUP x n)
v = n x u
```

For a right-handed camera, the camera usually looks along -n.

The camera basis is commonly written as three perpendicular unit vectors:

- `u`: camera's local right direction
- `v`: camera's local up direction
- `n`: camera's backward direction

In a common right-handed viewing convention, the camera looks along the `−n` direction.

---

4. Building the View Coordinate System

Suppose:

- `eye` is the camera position,
- `center` is the look-at point,
- `VUP` is the approximate view-up direction.

A common construction is:

$$n = \frac{eye - center}{\|eye - center\|}$$

$$u = \frac{VUP \times n}{\|VUP \times n\|}$$

$$v = n \times u$$

Here:

- `n` points from the target back toward the eye,
- `u` points to the camera's right,
- `v` points upward in the camera view.

Figure 4: The LookAt Matrix Concept

Input camera data

```
eye = camera position
center = target point
VUP = approximate up direction
```

Build an orthonormal basis

```
n = normalize(eye - center)
u = normalize(VUP x n)
v = n x u
```

The view matrix stores dot products with u, v, n, and subtracts the eye position by translation.

Column-vector convention:

`P_view = V P_world, so the view transform is applied before projection.`

Important Intuition

The camera transform does not move the camera through the world in the ordinary sense. Instead, it transforms the entire world so that the camera appears to be at the origin.

This is similar to sitting in a moving car. From your point of view, you are stationary and the world moves around you.

---

5. World-to-View Transformation

After the camera basis is created, world-space points are converted to view-space points.

Using column-vector convention:

$$P_{view} = VP_{world}$$

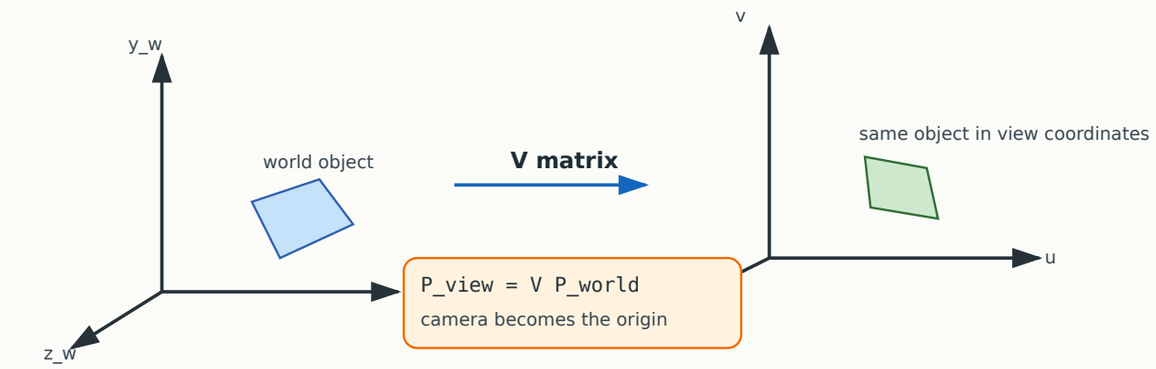
where `V` is the view matrix.

A common view matrix has the form:

$$V = \begin{bmatrix} u_x & u_y & u_z & -u \cdot eye \\ v_x & v_y & v_z & -v \cdot eye \\ n_x & n_y & n_z & -n \cdot eye \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This matrix expresses the point's coordinates relative to the camera axes.

Figure 3: World-to-View Transformation



Worked Example 1: Simple Camera

Let:

$$eye = (0, 0, 5), \quad center = (0, 0, 0), \quad VUP = (0, 1, 0)$$

Then:

$$n = normalize((0, 0, 5) - (0, 0, 0)) = (0, 0, 1)$$

$$u = normalize((0, 1, 0) \times (0, 0, 1)) = (1, 0, 0)$$

$$v = n \times u = (0, 1, 0)$$

So the camera is looking down the negative z direction. A point at the world origin appears 5 units in front of the camera.

---

6. View Volume

The **view volume** is the region of 3D space visible to the camera. Geometry outside this region is removed or clipped.

There are two common view volumes:

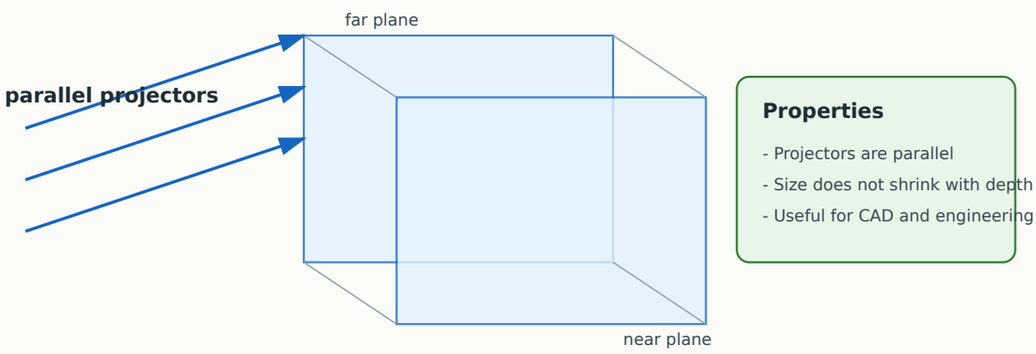
1. Orthographic view volume
2. Perspective view frustum

---

7. Orthographic View Volume

In orthographic projection, projectors are parallel. The view volume is a rectangular box.

Figure 5: Orthographic View Volume



Orthographic projection is useful when accurate measurements are important. For example:

- engineering drawings,
- architectural layouts,
- CAD systems,
- technical diagrams.

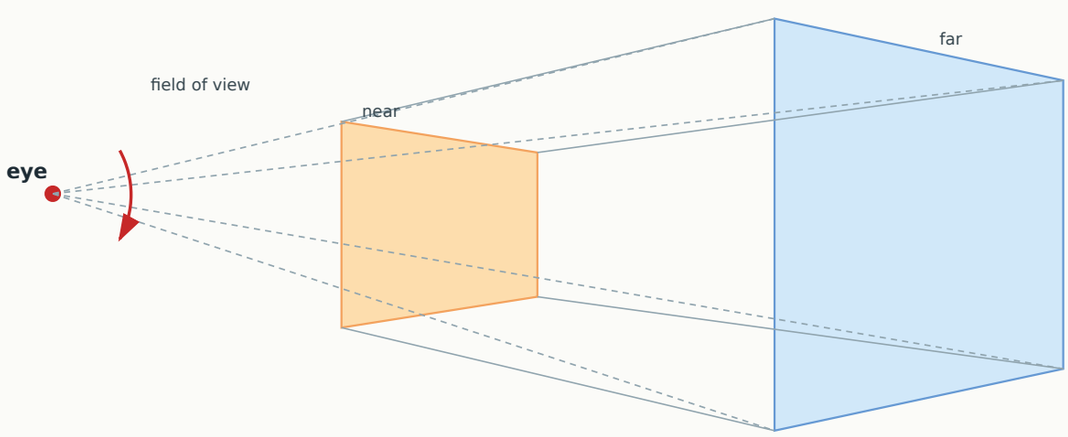
In orthographic projection, an object does not become smaller simply because it is farther from the camera.

---

8. Perspective View Frustum

In perspective projection, all projectors meet at the eye point. The view volume is a truncated pyramid called a **frustum**.

Figure 6: Perspective View Frustum



A perspective view volume is a truncated pyramid: objects farther away project s

Perspective projection is useful for realistic rendering because far objects appear smaller.

The frustum is bounded by six planes:

1. left plane,
2. right plane,
3. bottom plane,
4. top plane,
5. near plane,
6. far plane.

Objects outside these planes are not visible.

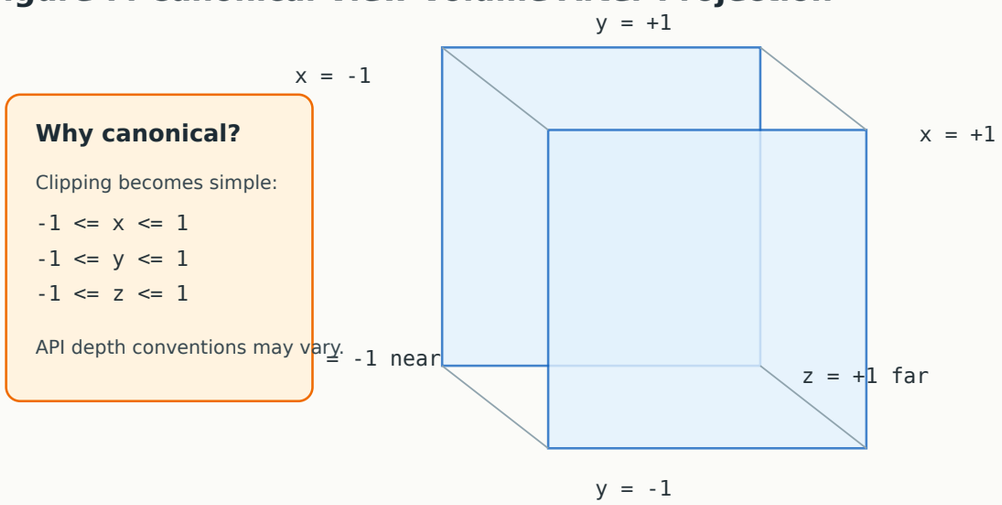
---

9. Canonical View Volume

Different cameras can have different positions, directions, field of view values, aspect ratios, and near/far distances. Clipping directly in every possible frustum would be inconvenient.

So graphics systems often transform the view volume into a standard shape called the **canonical view volume**.

Figure 7: Canonical View Volume After Projection



A common canonical volume is:

$$-1 \leq x \leq 1$$
$$-1 \leq y \leq 1$$
$$-1 \leq z \leq 1$$

Different graphics APIs may use different depth conventions, but the idea is the same: transform the visible region into a simple standard region.

---

10. What Is 3D Clipping?

Clipping removes parts of geometric objects that lie outside the view volume.

Clipping can apply to:

- points,
- line segments,

- polygons,
- curves,
- surfaces after tessellation.

The purpose is not just efficiency. Clipping is also needed for correctness. A line partly behind the camera or crossing the near plane can produce invalid projection results if not handled properly.

---

11. Point Clipping in 3D

For a canonical view volume, point clipping is simple.

A point `(x, y, z)` is inside if:

$$-1 \le x \le 1$$
$$-1 \le y \le 1$$
$$-1 \le z \le 1$$

If any condition fails, the point is outside.

Example

Check the point:

$$P = (0.4, -0.2, 0.7)$$

Since all three coordinates are between `-1` and `1`, the point is inside.

Check another point:

$$Q = (1.3, 0.2, 0.5)$$

Here `x = 1.3`, which is outside the allowed range. So `Q` is outside the right clipping plane.

---

12. Clipping Plane and Inside Test

A clipping plane can be written as:

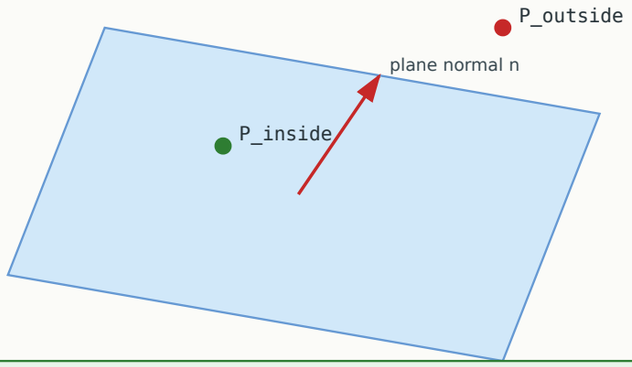
$$Ax + By + Cz + D = 0$$

For a point `P=(x,y,z)`, compute:

$$f(P) = Ax + By + Cz + D$$

The sign of `f(P)` tells which side of the plane the point lies on.

Figure 8: Clipping Plane and Inside Test




```
Plane equation: A x + B y + C z + D = 0
Inside test: evaluate f(P) = A x_P + B y_P + C z_P + D
The sign of f(P) tells whether the point is on the allowed side of the plane.
```

Depending on how the plane normal is defined, one side is considered inside and the other side is considered outside.

---

13. Line Clipping in 3D

A line segment between two endpoints can be written parametrically as:

$$P(t) = P_0 + t(P_1 - P_0), \quad 0 \le t \le 1$$

If a line crosses a clipping plane, the intersection point occurs where:

$$Ax(t) + By(t) + Cz(t) + D = 0$$

A compact formula is:

$$t = \frac{-f(P_0)}{f(P_1) - f(P_0)}$$

where:

$$f(P) = Ax + By + Cz + D$$

After finding `t`, substitute it back into `P(t)` to get the intersection point.

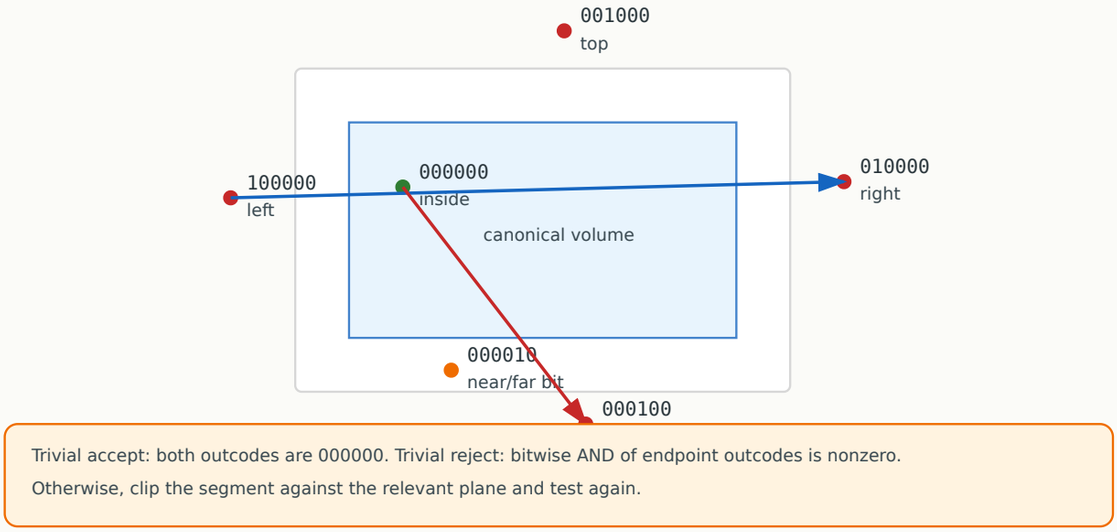
---

14. 3D Outcodes

The 2D Cohen-Sutherland line clipping idea can be extended to 3D. Instead of four boundary bits, 3D uses six bits:

- left,
- right,
- bottom,
- top,
- near,
- far.

Figure 9: 3D Outcodes for Line Clipping



For a line segment:

- If both endpoint outcodes are `000000`, the line is completely inside.
- If the bitwise AND of the two outcodes is nonzero, the line is completely outside on the same side of at least one plane.
- Otherwise, the line may cross the view volume and must be clipped.

Worked Example 2: Trivial Reject

Suppose endpoint A is outside the left plane and endpoint B is also outside the left plane.

Then both outcodes have the left bit set.

The bitwise AND is nonzero, so the entire line is outside and can be rejected.

Worked Example 3: Possible Crossing

Suppose endpoint A is outside the left plane, but endpoint B is inside.

The bitwise AND is zero, so the line may cross the view volume. We clip it against the left plane and keep the visible part.

---

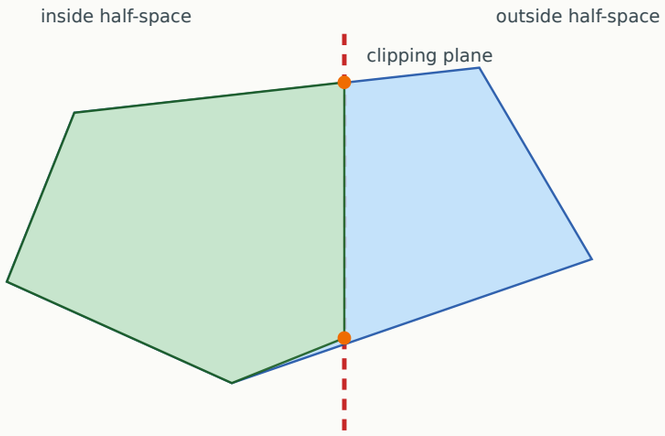
15. Polygon Clipping in 3D

A polygon may be partly inside and partly outside a clipping plane. Polygon clipping creates a new polygon containing only the visible part.

A common approach is similar to the Sutherland-Hodgman algorithm:

1. Clip the polygon against one plane.
2. Use the result as input for the next plane.
3. Repeat for all six clipping planes.

Figure 10: Polygon Clipping Against One Plane



Sutherland-Hodgman idea: process every edge; keep inside vertices and create intersection po

For each polygon edge, there are four cases:

Start vertex	End vertex	Output
inside	inside	end vertex
inside	outside	intersection point
outside	inside	intersection point and end vertex
outside	outside	nothing

This is the same conceptual idea as 2D polygon clipping, but the clipping boundaries are planes instead of lines.

---

16. Why Clip Before the Perspective Divide?

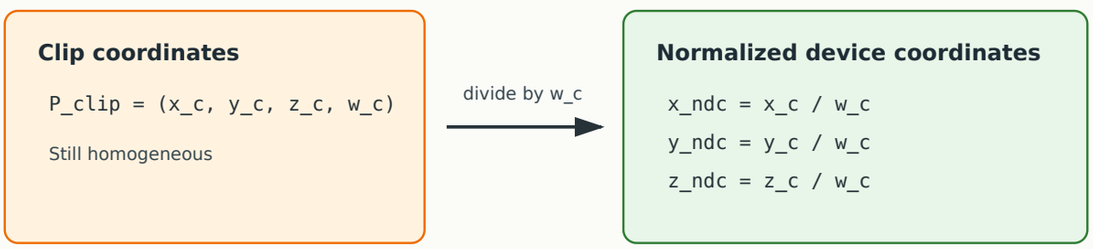
After projection, points are in homogeneous clip coordinates:

$$P_{clip} = (x_c, y_c, z_c, w_c)$$

To get normalized device coordinates, we divide by `w_c`:

$$x_{ndc} = \frac{x_c}{w_c}, \quad y_{ndc} = \frac{y_c}{w_c}, \quad z_{ndc} = \frac{z_c}{w_c}$$

Figure 11: Perspective Divide



Important: clipping is usually done before the divide so that geometry crossing the view volume is ha

However, if geometry crosses the near plane or goes behind the camera, division by `w_c` can create invalid or unstable results.

Therefore, clipping is normally performed before the perspective divide.

In homogeneous clipping, the common idea is:

$$-w_c \le x_c \le w_c$$
$$-w_c \le y_c \le w_c$$
$$-w_c \le z_c \le w_c$$

This keeps only geometry inside the clip volume before converting to normalized coordinates.

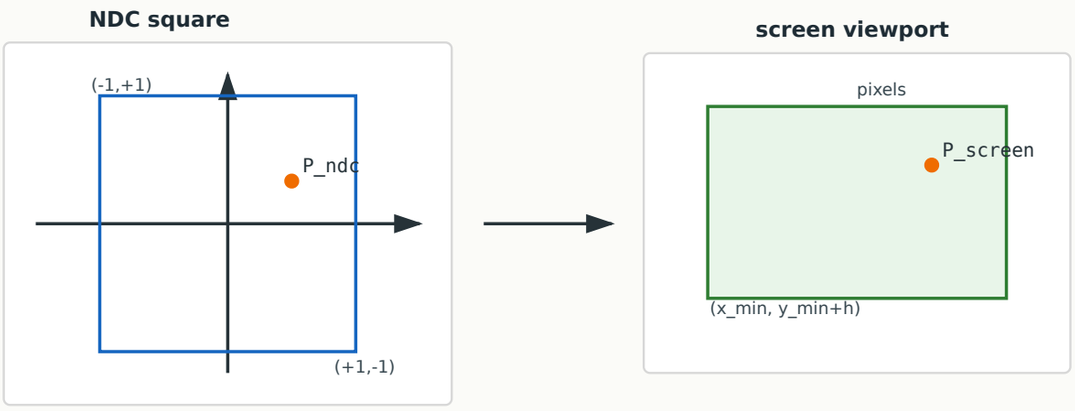
---

17. Viewport Mapping

After clipping and perspective divide, points are in normalized device coordinates. These are not yet screen pixels.

The final step is **viewport mapping**.

Figure 12: Mapping NDC to the Screen Viewport



For a viewport with top-left corner `(x_min, y_min)`, width `w`, and height `h`, a common screen mapping is:

$$x_s = x_{min} + \frac{(x_{ndc} + 1)w}{2}$$
$$y_s = y_{min} + \frac{(1 - y_{ndc})h}{2}$$

The y formula may look reversed because many screen coordinate systems have y increasing downward.

Worked Example 4: Viewport Mapping

Suppose:

- viewport size = `800 x 600`,
- top-left corner = `(0,0)`,
- normalized point = `(0.5, -0.25)`.

Then:

$$x_s = 0 + \frac{(0.5 + 1)800}{2} = 600$$

$$y_s = 0 + \frac{(1 - (-0.25))600}{2} = 375$$

So the screen point is:

$$(600, 375)$$

---

18. Full 3D Viewing Pipeline

A simplified pipeline is:

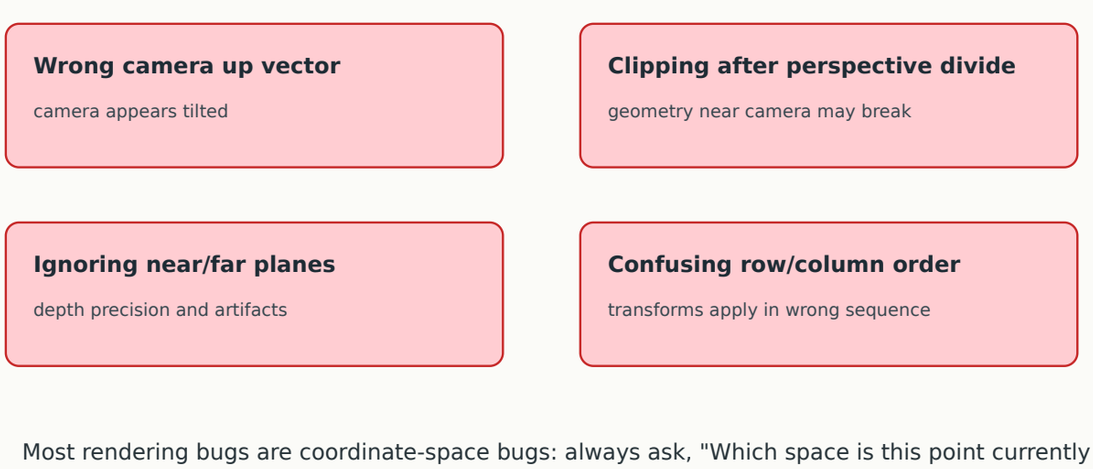
1. Define geometry in object coordinates.
2. Transform objects to world coordinates.
3. Transform world coordinates to view coordinates.
4. Apply projection to get clip coordinates.
5. Clip against the view volume.
6. Perform perspective divide.
7. Map normalized coordinates to viewport pixels.
8. Rasterize primitives.

The important thing is to track the coordinate space at every step.

---

19. Common Mistakes

Figure 13: Common 3D Viewing Mistakes



Most rendering bugs are coordinate-space bugs: always ask, "Which space is this point currently in?"

Mistake 1: Confusing Camera Movement and World Movement

Moving the camera right is equivalent to transforming the world left in view coordinates.

Mistake 2: Applying Transformations in the Wrong Order

With column vectors, the rightmost matrix is applied first.

For example:

$$P' = P_{proj}VMP$$

means the model transform `M` is applied first, then the view transform `V`, then projection.

Mistake 3: Ignoring the Near Plane

The near plane is not just an optimization. It prevents problematic projection of geometry too close to or behind the camera.

Mistake 4: Forgetting API Conventions

Different systems may use different handedness and depth ranges. Always know the convention being used.

---

20. Applications

3D viewing and clipping are used in almost every 3D graphics system.

20.1 Games

A game camera continuously changes position and direction. The engine clips geometry outside the view frustum before drawing.

20.2 CAD and Engineering

Orthographic views allow accurate front, side, and top views.

20.3 Medical Visualization

3D scans can be viewed from different angles. Clipping planes can cut through a volume to inspect internal structures.

20.4 Virtual Reality

Each eye has a slightly different view frustum. Correct viewing transforms are essential for depth perception.

20.5 Robotics and Simulation

Virtual cameras are used to simulate what a robot would see from a particular viewpoint.

---

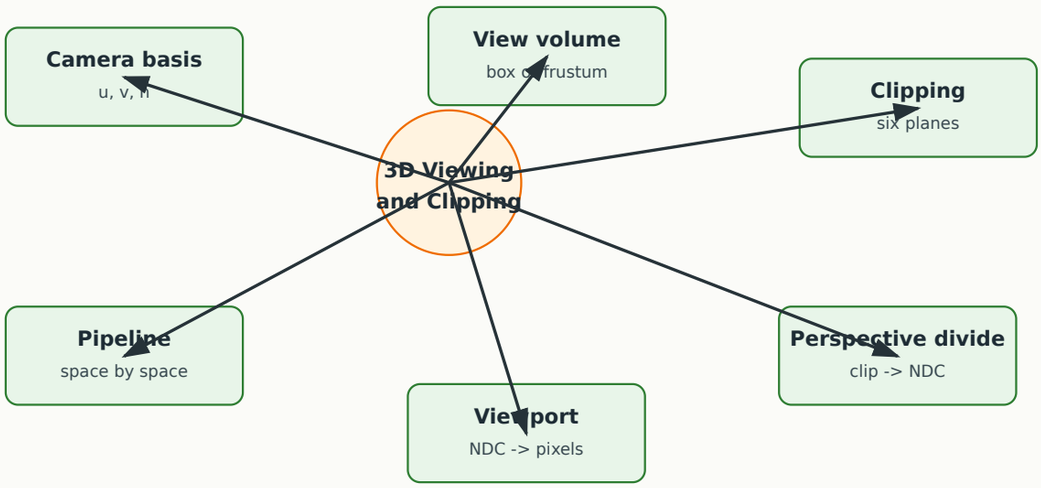
21. Trivia

- Early 3D graphics systems often spent significant time clipping lines and polygons before rasterization because drawing outside the visible region wasted computation.
- The term **frustum** means a pyramid or cone with its top cut off.
- Camera conventions differ across systems. Some systems use a right-handed view coordinate system, while others use a left-handed one.
- The near plane strongly affects depth-buffer precision. A very small near-plane distance can make depth artifacts worse.

---

22. Lecture Summary

Figure 14: Lecture 9 Summary Map



Key ideas:

- 3D viewing defines how the camera sees the world.
- A camera coordinate system is built from eye, look-at point, and up vector.
- The view volume defines what is visible.
- Orthographic projection uses a box-shaped view volume.
- Perspective projection uses a frustum.
- Clipping removes geometry outside the view volume.
- 3D line clipping can use six outcode bits.
- Polygon clipping processes edges against each clipping plane.
- Perspective divide converts homogeneous clip coordinates to normalized device coordinates.
- Viewport mapping converts normalized coordinates to screen pixels.

---

23. Quick Concept Checks

1. What is the purpose of the view transformation?
2. Why is the camera often moved to the origin in view coordinates?
3. What is the difference between an orthographic view volume and a perspective frustum?
4. Why are there six clipping planes in 3D viewing?
5. What does an outcode of `000000` mean?
6. Why is clipping normally performed before perspective divide?
7. Why can the screen y-coordinate formula look reversed?

---

24. Exam-Style Questions

Short Questions

1. Define view volume in 3D graphics.
2. Explain the purpose of the near and far clipping planes.
3. What is a perspective frustum?
4. What is meant by normalized device coordinates?
5. Explain why viewport mapping is needed after projection.

Analytical Questions

1. A point in NDC is `(0.25, 0.5)`. Map it to a viewport of size `1000 x 800` with top-left origin `(0,0)`.
2. A line segment has both endpoints outside the right clipping plane. What can be concluded using the outcode test?
3. Explain why clipping a polygon against six planes may create new vertices.
4. A virtual camera is placed at `(0,0,10)` and looks at the origin. What is the intuitive meaning of the view transformation?
5. Why may geometry behind the camera create problems if perspective divide is performed before clipping?

---

25. Suggested Reading / Preparation for Next Lecture

Before the next lecture, review:

- depth comparison,
- z-buffer idea,
- visibility of surfaces,
- difference between clipping and hidden surface removal.

The next lecture will move from **what is inside the camera view** to **which surfaces are actually visible after projection**.